with the password awesome.)

Click “Edit” on any of the concerts. You should see the form display inline, and submitting the form should result in the updated data appearing back inline.

This works. We now have inline editing with no custom JavaScript written. We’ve barely even written any Rails; mostly, we’ve just added a couple of tags to the HTML output we were already generating.

What’s happening here?

By putting our HTML inside the <turbo-frame> tag, we are allowing Turbo to control both the request to the server and the response from the server.

When we click the Edit button, we make an ordinary Rails GET request—in this case for the ConcertController#edit action. Rails handles this request normally (almost). If you look in the browser network tab, you can see that the return value is the entire edit page, including the <h1>Editing Concert</h1> header. Turbo inserts a Turbo-Frame header into the request, and Rails recognizes that header and optimizes by not rendering the layout in the return value, since presumably the part that the receiving Turbo Frame is interested in is not in the layout.

When the response comes back to the browser, Turbo intercepts it and looks for the part of the response that contains a Turbo Frame tag with the matching ID. This is why the Turbo Frame in the form partial and one in the display partial had to have the same DOM ID. Turbo extracts that Turbo Frame from the response and replaces the existing Turbo Frame’s contents with the contents from the incoming response’s Turbo Frame, meaning we get the form, which is inside the frame, but not the header, which is outside the frame. The Network tab of your browser tools is useful here; it can show the headers in the request and also show you the full response from the server, including the parts that Turbo Frame doesn’t use.

If the incoming response does not contain a matching Turbo Frame, the existing Turbo Frame is replaced with blank contents. It seems to disappear, though you